IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

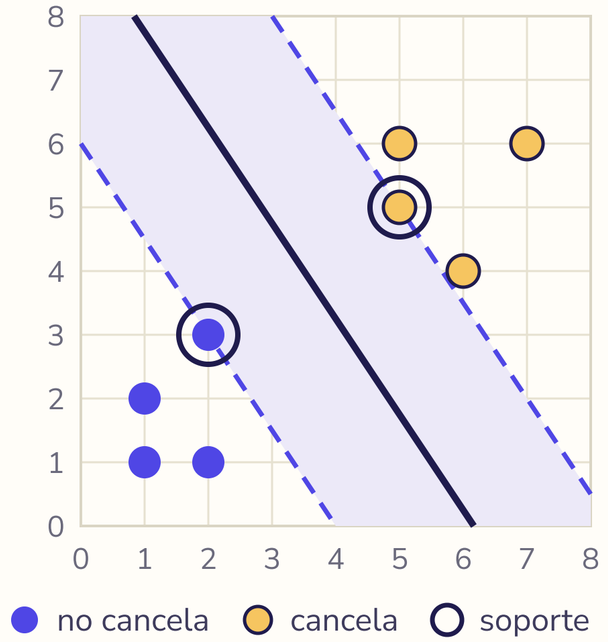
Máquinas de vectores de soporte (SVM)

Teoría, práctica y, al final, material de consulta

Sprint 8 · S08_04 · teoría breve (10 min el 12/11), 40 min de práctica en casa y material de consulta

LA IDEA

La calle más ancha entre dos grupos



Frontera

La línea del centro de la calle: a un lado se predice una clase, al otro, la otra.

Margen

El ancho de la calle, aquí **3,61**. La SVM busca la calle **más ancha** posible.

Vectores de soporte

Los puntos que tocan las aceras, aquí **(2, 3)** y **(5, 5)**: ellos sujetan la calle.

LA IDEA

Solo mandan los vectores de soporte

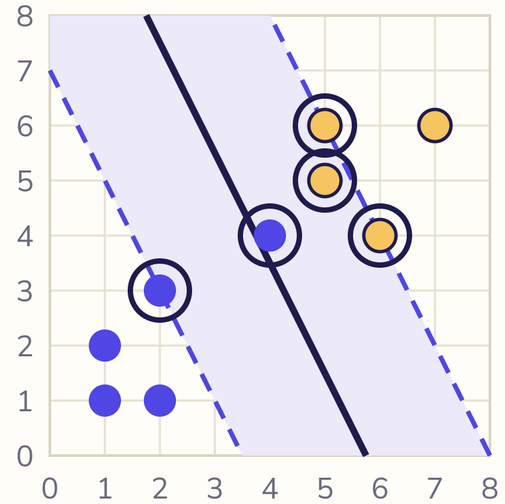
Qué hago con los 8 puntos	¿Cambia la recta?
Quito uno de los 6 puntos lejanos	**No**: las 6 pruebas dan la misma recta
Muevo el punto (2, 3) a (1, 3)	**Sí**: de 6x + 4y = 37 pasa a 2x + y = 10
Quito el punto (2, 3)	**Sí**: cambia la recta y ahora hay 4 vectores de soporte

Como en una tienda de campaña: solo los palos que la sujetan importan, el resto de la tela no.

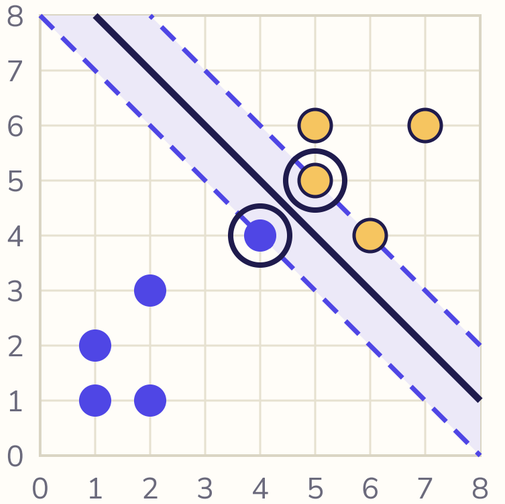
LA IDEA

C: cuánto cuesta equivocarse

C = 0,1 · calle ancha



C = 100 · calle estrecha



C = 0,1 (pequeño)

Margen **4,03** · 1 error en train · 5 vectores de soporte

C = 100 (grande)

Margen **1,41** · 0 errores en train · 2 vectores de soporte

C grande: calle estrecha, ajusta mucho (riesgo de sobreajuste). C pequeño: calle ancha, tolera errores.

LA IDEA

Cuando una recta no basta: el kernel

x	−4	−3	−2	−1	0	1	2	3	4
x^2	16	9	4	1	0	1	4	9	16
clase	0	0	0	1	1	1	0	0	0

Kernel = curvar el espacio. El RBF lo hace solo, sin construir columnas, y deja fronteras curvas.

KERNEL RBF

gamma: hasta dónde llega cada punto

gamma	Exactitud en train	Exactitud en test	Puntos guardados	Qué pasa
0,1	0,80	0,77	201	**Infraajusta**: frontera casi recta
1	0,85	0,84	104	Equilibrio
100	0,96	0,74	196	**Sobreajusta**: islotes alrededor de cada punto

gamma bajo: influencia lejana, frontera suave. gamma alto: influencia local, frontera retorcida.

Con gamma = 100 guarda 196 de 210 puntos: casi todos. **La diferencia entre train y test** es la señal de sobreajuste.

PREPARAR LOS DATOS

Escalar es obligatorio: la SVM mide distancias

Reserva	precio_noche	noches
A	80	2
B	90	7
C	200	2

Pareja	Distancia sin escalar	Escalando
A – B	11,18	2,13
A – C	120,00	2,21
B – C	110,11	2,93

Sin escalar, A y B parecen casi iguales pese a 5 noches de diferencia: manda el precio, que es la cifra más grande.

AJUSTE DE PARÁMETROS

Probar C y gamma con la rejilla

C \ gamma	0,1	1
0,1	0,795	0,840
1	0,835	**0,860**
10	0,835	0,825

6 combinaciones × 5 pliegues

= **30 ajustes**, y uno más al final con la mejor combinación.

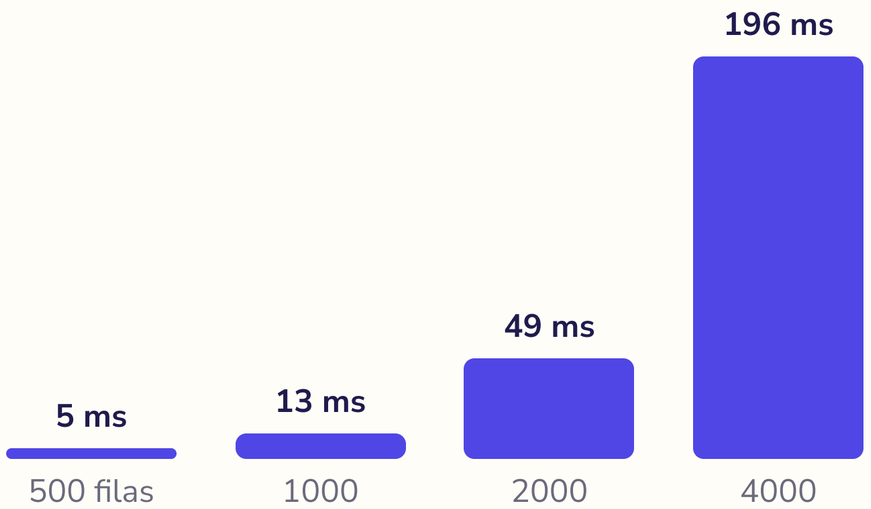
Cuidado con los empates

Las cuatro mejores están a solo 0,025; entre pliegues, la nota varía unos 0,06.

Se prueban todas las combinaciones con validación cruzada y se elige la de mejor media.

COSTE Y LÍMITES

Con muchas filas, la SVM se hace lenta



×4 filas, ×15 tiempo

De 1000 a 4000 filas, el tiempo se multiplicó por **14 a 21** en siete pruebas: crece casi como el cuadrado de las filas.

predict_proba pide **probability=True**

Da probabilidades, pero entrena unas **5 veces** más despacio. Para ordenar basta **decision_function**.

¿5 millones de reservas? Mejor LinearSVC o un boosting: la SVM RBF no escala.

PRÁCTICA · CUADERNO S08_04 (CASA)

Lo que haremos en el cuaderno

Parte	Qué haremos	Dónde
Preparación	Cargar las herramientas y las dos lunas	Casa
Ejercicio 1	SVM lineal frente a SVM RBF en las dos lunas	Casa
Ejemplo	El efecto de gamma en la frontera	Casa
Ejercicio 2	El AUC con y sin escalado	Casa
Ejercicio 3	Buscar los mejores C y gamma	Casa
Ejemplo	Mapa de calor del AUC según C y gamma	Casa
Ejercicio 4	Cuánto crece el tiempo con el número de filas	Casa
Mini-reto	Lineal, RBF por defecto y RBF ajustada	Casa

Cada ejercicio: **lee** → **escribe** → **ejecuta** → **comprueba el ✔**. La solución está plegada: ábrela tras intentarlo.

CONSULTA

Material de consulta: código y aclaraciones

Cuándo abrirlo

Después de intentar el ejercicio, o cuando algo no te salga. En clase no se proyecta.

Cómo está ordenado

En el **mismo orden que el cuaderno**: ejercicios 1 a 4 y mini-reto. Cada uno trae su código y sus aclaraciones.

Ejercicio	Tema	Diapositivas
1	Lineal frente a RBF	12 y 13
2	Con y sin escalado	14 y 15
3	Buscar C y gamma	16 y 17
4	Tiempo según las filas	18 y 19
Mini-reto	Tres SVM	20 y 21

CONSULTA · EJERCICIO 1

SVM lineal frente a RBF (ejercicio 1)

```
svm_lineal = SVC(kernel="linear", C=1).fit(Xm_train, ym_train)
svm_rbf = SVC(kernel="rbf", C=1, gamma=1).fit(Xm_train, ym_train)
acc_lineal = svm_lineal.score(Xm_test, ym_test)    # 0.81
acc_rbf = svm_rbf.score(Xm_test, ym_test)          # 0.91
n_soporte_rbf = int(svm_rbf.n_support_.sum())      # 91
```

0,81

exactitud lineal

0,91

exactitud RBF

La RBF guarda 91 de 280 puntos: un tercio decide la frontera.

CONSULTA · ACLARACIONES · EJERCICIO 1

Leer lineal frente a RBF (ejercicio 1)

kernel="linear" frente a **"rbf"**

El lineal traza una **recta**; el RBF, una frontera **curva** que sigue la forma de las lunas.

n_support_.sum()

Cuenta los **vectores de soporte** de las dos clases: los puntos que sujetan la frontera.

0,81 frente a 0,91

Las dos lunas **no se separan con una recta**: la lineal rinde peor. 91 de 280 puntos (32 %) definen la frontera.

Efecto de **gamma**

0,05: casi recta (test 0,81) · 1: curva suave (0,91) · 50: islotes (train 0,95, test 0,92): **memoriza**.

CONSULTA · EJERCICIO 2

El AUC con y sin escalado (ejercicio 2)

```
svm_rbf_res = Pipeline([("prep", prep_escalado()), ("svm", SVC(kernel="rbf", C=1))]).fit(X_train, y_train)
svm_rbf_sin = Pipeline([("prep", prep_sin_escalar()), ("svm", SVC(kernel="rbf", C=1))]).fit(X_train, y_train)
auc_con_escalado = roc_auc_score(y_test, svm_rbf_res.decision_function(X_test))    # 0.735
auc_sin_escalado = roc_auc_score(y_test, svm_rbf_sin.decision_function(X_test))    # 0.671
```

0,735

AUC con escalado

0,671

AUC sin escalar

Sin escalar se pierden **0,064** de AUC.

CONSULTA · ACLARACIONES · EJERCICIO 2

Leer el efecto del escalado (ejercicio 2)

prep_escalado() y prep_sin_escalar()

Los dos **imputan y codifican**; solo el primero **escala** las numéricas.

decision_function(X_test)

Da una **puntuación** por reserva (distancia a la frontera): sirve para el AUC sin pedir probabilidades.

0,735 frente a 0,671

Sin escalar se pierden **0,064** de AUC: un modelo claramente peor.

Por qué es obligatorio

La SVM trabaja con **distancias**: **antelacion_dias** y **precio_noche** (valores grandes) aplastan a las pequeñas.

CONSULTA · EJERCICIO 3

Buscar los mejores C y gamma (ejercicio 3)

```
busqueda = GridSearchCV(Pipeline([("prep", prep_escalado()), ("svm", SVC(kernel="rbf"))]),
                        rejilla, cv=cv, scoring="roc_auc").fit(sub_X, sub_y)
mejores_params = busqueda.best_params_    # {'svm__C': 10, 'svm__gamma': 0.003}
mejor_auc_cv = busqueda.best_score_       # 0.752
```

C = 10 · gamma = 0,003

mejores parámetros

0,752

AUC en validación cruzada

CONSULTA · ACLARACIONES · EJERCICIO 3

Leer la búsqueda de parámetros (ejercicio 3)

GridSearchCV(..., rejilla, cv=cv)

Entrena **cada combinación** de la rejilla con validación cruzada: 9 combinaciones × 3 folds = **27 modelos** (más un reajuste).

Submuestra de 1500

La SVM **tarda mucho con muchas filas** y la búsqueda repite el ajuste: con 1500 el resultado es orientativo y rápido.

Mapa de calor 3 × 3

Mejor: **C = 10**, **gamma = 0,003** (**0,752**). Peor: **C = 10**, **gamma = 0,03** (**0,713**): C y gamma altos = sobreajuste.

Qué importa más

Del mejor al peor, solo **0,04**: ajustar parámetros ayuda menos que **escalar** (0,064).

CONSULTA · EJERCICIO 4

Cuánto crece el tiempo con las filas (ejercicio 4)

```
Z = prep_escalado().fit_transform(X_train)
yz = y_train.to_numpy()
tamanos = [500, 1000, 2000, 4000]
segundos = []
for n in tamanos:
    t0 = time.perf_counter()
    SVC(kernel="rbf", C=1, gamma=0.01).fit(Z[:n], yz[:n])
    segundos.append(time.perf_counter() - t0)
factor_x4 = segundos[-1] / segundos[1]    # 19.2
```

0,01 · 0,02 · 0,09 · 0,47

segundos con 500, 1000, 2000 y 4000 filas

19× tiempo

4× filas

CONSULTA · ACLARACIONES · EJERCICIO 4

Leer el crecimiento del tiempo (ejercicio 4)

segundos[-1] / segundos[1]

El último tiempo (4000 filas) entre el segundo (1000 filas): **4× filas**. Al revés daría menos de 1.

Z[:n], yz[:n]

Las **n primeras** filas escaladas y sus etiquetas: el **append** va **dentro** del **for**, una vez por tamaño.

19 veces, no 4

Con tiempo lineal serían 4 veces. Crece **más que linealmente** (casi con el cuadrado, que sería 16).

Con 1 millón de filas

Inviable: 250 veces más filas que 4000 tardarían decenas de miles de veces más. Mejor **LinearSVC** o boosting.

CONSULTA · MINI-RETO

Tres SVM frente a frente (mini-reto)

```
variantes = {"lineal": SVC(kernel="linear", C=1), "rbf_defecto": SVC(kernel="rbf", C=1),
             "rbf_ajustada": SVC(kernel="rbf", C=C_aj, gamma=g_aj)}
for nombre, svm in variantes.items():
    pipe = Pipeline([("prep", prep_escalado()), ("svm", svm)]).fit(X_train, y_train)
    # auc_test con decision_function · n_soporte = svm.n_support_.sum() · segundos de ajuste
```

Modelo	AUC en test	Vectores de soporte	% del train
lineal	0,743	3327	69 %
rbf_defecto	0,735	3515	73 %
rbf_ajustada	**0,744**	3374	70 %

La lineal empata con la RBF ajustada y es más sencilla.

CONSULTA · ACLARACIONES · MINI-RETO

Leer la comparativa (mini-reto, casa)

mejores_params

Llevan el prefijo **svm__** (**svm__C**): hay que **quitarlo** para usarlos en **SVC(C=..., gamma=...)**.

n_support_.sum()

Los vectores de soporte son **entre el 69 % y el 73 %** de las 4800 reservas: el modelo guarda casi todo el entrenamiento.

Lineal ≈ RBF ajustada

0,743 y 0,744 casi empatan; la RBF por defecto (0,735) queda detrás.

Qué elegirías

La **lineal**: mismo AUC, más sencilla (sin **gamma**) y sin buscar parámetros.